An Optimization-Based Motion Planner for Autonomous Maneuvering of Marine Vessels in Complex Environments *

Kristoffer Bergman † Oskar Ljungqvist † Jonas Linder ‡ Daniel Axehill †

arXiv:2005.02674v1 [math.OC] 6 May 2020

***Abstract* —The task of maneuvering ships in confined environments is a difficult task for a human operator. One major reason is due to the complex and slow dynamics of the ship which need to be accounted for in order to successfully steer the vehicle. In this work, a two-step optimization-based motion planner is proposed for autonomous maneuvering of ships in constrained environments such as harbors. A lattice-based motion planner is used in a first step to compute a feasible, but suboptimal solution to a discretized version of the motion planning problem. This solution is then used to enable efficient warm-start and as a terminal manifold for a second receding-horizon improvement step. Both steps of the algorithm use a high-fidelity model of the ship to plan feasible and energy-efficient trajectories. Moreover, a novel algorithm is proposed for automatic computation of spatial safety envelopes around the trajectory computed by the lattice-based planner. These safety envelopes are used in the second improvement step to obtain collision-avoidance constraints which complexity scales very well with an increased number of surrounding obstacles. The proposed optimization-based motion planner is evaluated with successful results in a simulation study for autonomous docking problems in a model of the Cape Town harbor.**

1 Introduction

Autonomous ship technology is currently witnessing an increased interest in order to improve efficiency and safety of future sea transports. Historically, complex and precise maneuvering of ships in harbors, e.g., docking, required external help from support vessels such as tug boats. In recent years, as modern ships have been equipped with more thrusters with better capabilities, the ships' ability to independently perform advanced maneuvers has increased. Meanwhile, to aid humans operating modern ships equipped with high degrees of redundancy in their truster setup, various control systems have been developed. These systems are designed to alleviate the need of manually allocating thruster commands Fossen [2011], and to automatically maintain a desired course and set speed or vehicle pose (*dynamic positioning*) Perez [2006], Veksler et al. [2016]. As a consequence, the current trends in advanced motion control of ships are moving towards an increased degree of automation.

Although a large amount of different feedback-control techniques has been proposed in the literature for marine vessels, see, e.g., Fossen [2011], Veksler et al. [2016], Perez [2006], Moreira et al. [2007], there only exists a limited amount of work that focus on simplified versions of the trajectory-planning problem in complex and constrained environments, e.g., Martinsen et al. [2019], Du et al. [2018]. Compared to agile unmanned surface vehicles (USVs), where maritime collision avoidance maneuvers can be computed by partially or completely neglecting the system dynamics Han et al. [2020], Kufoalor et al. [2019], reliable maneuvering of ships requires a motion planner that accounts for the ship's dynamics and physical constraints in order to compute feasible, safe and optimized trajectories Martinsen et al. [2019].

In this work, a two-step optimization-based motion planning framework is proposed for automatic maneuvering of ships in cluttered environments such as harbors. The framework is based on techniques presented in our recent work Bergman et al. [June 2019,a,b] and is here tailored and extended to account for the specific needs related to ships. A lattice-based motion planning algorithm Pivtoraiko et al. [2009] is used in a first step to compute a resolution optimal solution to a discretized version of the motion planning problem. By employing classical graph-search algorithms, the lattice-based planner concatenates motion segments from a finite library of optimized maneuvers Bergman et al. [June 2019] and is responsible for solving combinatorial aspects of the motion planning problem, such as selecting which side to pass an obstacle. Due to its deterministic properties and efficiency, lattice-based motion planners have been used with great success for a variety of different vehicular systems Pivtoraiko et al. [2009], Andersson et al. [2018], Ljungqvist et al. [2019], Likhachev and Ferguson [2009], Cirillo et al. [2014]. Similar motion planning techniques have also been developed for

*This work was partially supported by FFI/VINNOVA and the Wallenberg Artificial Intelligence, Autonomous Systems and Software Program (WASP) funded by Knut and Alice Wallenberg Foundation.

†Kristoffer Bergman, Oskar Ljungqvist and Daniel Axehill are with the Division of Automatic Control, Linköping University, Linköping, Sweden (e-mail: {kristoffer.bergman, oskar.ljungqvist, daniel.axehill}@liu.se)

‡Jonas Linder is at ABB AB, Corporate Research, Västerås, Sweden (e-mail: jonas.x.linder@se.abb.com)

USVs Du et al. [2018] using a library of basic curve segments. However, since a lattice-based planner is restricted to use a discretized search space, the computed solutions often suffer from discretization artifacts Andreasson et al. [2015].

To improve the solution computed by the lattice-based planner, a second optimization-based improvement step is used that is warm-started with the trajectory computed by the lattice-based planner, similar to Bergman et al. [a]. The improvement step can either optimize the complete trajectory at once, or it can run in a receding-horizon fashion over a sliding time-window as proposed in Bergman et al. [b]. In the latter approach, the output computed by the lattice-based planner is used both for enabling an efficient warm-start and as a terminal manifold to ensure feasibility and convergence to the desired goal state.

The proposed receding-horizon improvement step shares similarities with the work in Martinsen et al. [2019], but instead of using a quadratic terminal cost to represent a possibly under-estimate of the remaining cost-to-go after the prediction horizon, the nominal solution computed by the lattice-based planner provides the receding horizon optimization-step with an over-estimate of the remaining cost-to-go. This means that feasibility can be ensured also in complex environments Bergman et al. [b]. Furthermore, compared to Martinsen et al. [2019] which restricts the ship's body to a manually selected convex polytope, a novel collision-avoidance algorithm is proposed to automatically compute spatial safety envelopes along each sample of the nominal solution computed by the lattice-based planner. These safety envelopes are described, e.g., using convex polytopes, and represent the constraints imposed on the ship's body from surrounding obstacles. Similar collision-avoidance techniques have been proposed in Liu et al. [2017], Schoels et al. to compute convex safety envelopes for drones and linear systems, respectively. These previously presented techniques can robustly be used for vehicles whose body can be reasonably described by a single bounding circle, i.e., not for elongated vehicles such as ships that are considered in this work. Compared to using individual bounding regions for each obstacle, as was done in Bergman et al. [a,b], the number of obstacle avoidance constraints using the proposed safety-envelope approach scales significantly better in the prediction horizon with an increased number of obstacles. Additionally, as the ship's body can be accurately approximated as a convex polytope Martinsen et al. [2019], a sufficient condition for collision avoidance at each time instance is to enforce that each vertex of the ship's body is within the computed convex safety envelope.

The remainder of the paper is organized as follows. In Section 2, the dynamic ship model and the motion planning problem are presented. The lattice-based motion planner is presented in Section 3. In Section 4, the collision-avoidance algorithm and the optimization-based receding-horizon improvement step are presented. Simulation results from autonomous docking in a model of the Cape Town harbor are presented in Section 5 and the paper is concluded in Section 6 with a summary of the contributions and a discussion of future research directions.

2 Ship modeling and problem formulation

In this section, the dynamic ship model used for motion planning is briefly described and the general motion planning problem is posed as an optimal control problem (OCP). The notation is adopted from Fossen [2011] and for more details regarding ship modeling, the reader is referred to, e.g., Fossen [2011], Perez [2006].

The ship is assumed to move on the ocean surface and it is thus sufficient to consider the horizontal 3 degrees of freedom motion. The motion of the ship is described using two coordinate systems: an Earth-fixed (inertial) system and a body-fixed system that is located at the center of mass of the ship. The Earth-fixed generalized position is given by $\boldsymbol{\eta} = \begin{bmatrix} x & y & \psi \end{bmatrix}^\mathsf{T} \in \mathbf{SE}(2)$ LaValle [2006] and the body-fixed generalized velocity vector is represented by $\boldsymbol{\nu} = \begin{bmatrix} u & v & r \end{bmatrix}^\mathsf{T} \in \mathbf{R}^3$, see Fig. 1. The generalized velocity and position is related through the kinematics

$$\dot{\boldsymbol{\eta}} = \boldsymbol{R}(\psi)\boldsymbol{\nu}, \tag{1}$$

where

$$\boldsymbol{R}(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{2}$$

is the rotation matrix. The kinetics, i.e., the motion induced by forces acting on the ship, is derived using rigid-body mechanics and theory of hydrodynamics, see Fossen [2011] for details. A model of the kinetics is given by

$$\boldsymbol{M}\dot{\boldsymbol{\nu}} + \boldsymbol{C}(\boldsymbol{\nu}) + \boldsymbol{D}(\boldsymbol{\nu}) = \boldsymbol{\tau}, \tag{3}$$

where $\boldsymbol{M}$ is the total inertia matrix, $\boldsymbol{C}(\boldsymbol{\nu})$ corresponds to Coriolis and centripetal forces, $\boldsymbol{D}(\boldsymbol{\nu})$ describes the damping and $\boldsymbol{\tau}$ describes the forces acting on the ship. Simplified models for the thrusters are used where it is assumed that forces induced by the propeller and rudder can be separated. The total forces can be written as $\boldsymbol{\tau} = \sum_{j=1}^{N_t} \boldsymbol{\tau}_j$, where N_t is the number of thrusters. The generalized forces of the jth thruster is assumed to be

$$\boldsymbol{\tau}_j = \underbrace{\begin{bmatrix} 1 & 0 & l_{y,j} \\ 0 & 1 & l_{x,j} \end{bmatrix}^\mathsf{T}}_{:=\boldsymbol{T}} \left(\theta_{p,j}\boldsymbol{\tau}_p(\alpha_j, n_j) + \theta_{r,j}\boldsymbol{\tau}_r(\alpha_j, \boldsymbol{\nu})\right), \tag{4}$$

where $l_{x,j}$ and $l_{y,j}$ describe the position of the thruster in the body-fixed coordinate system, $\theta_{p,j}$ is the propeller gain, $\theta_{r,j}$ is the rudder gain, α_j is the thruster angle and n_j is the propeller velocity. The propeller force is assumed to be quadratic with respect to the propeller velocity:

$$\boldsymbol{\tau}_p(\alpha_j, n_j) = \begin{bmatrix} \cos\alpha_j \\ \sin\alpha_j \end{bmatrix} |n_j| n_j, \quad j = 1, \ldots, N_t. \tag{5}$$

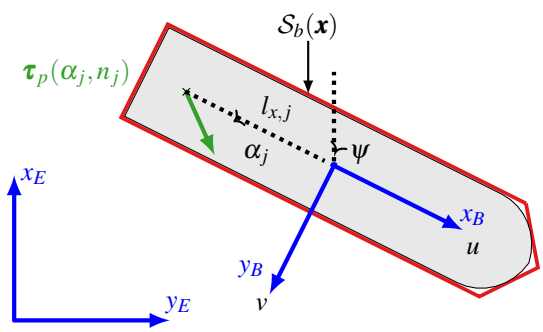


Figure 1: Definition of Earth-fixed (x_E, y_E) and body-fixed (x_B, y_B) coordinate systems, body-fixed velocities (u, v) and the convex polytope $\mathcal{S}_b(\boldsymbol{x})$ representing the bounding region of the ship's body at $\boldsymbol{x}$. Furthermore, the parameters for the jth thruster are illustrated.

The rudder force is assumed to be proportional to the square of the velocity V_j of the incoming water flow which is dependent on the thruster position and the ship velocity. The rudder force at the jth thruster is (see, e.g., Du et al. [2018] for details):

$$\boldsymbol{\tau}_r(\alpha_j, \boldsymbol{\nu}) = \begin{bmatrix} -\sin\alpha_j \\ \cos\alpha_j \end{bmatrix} F_N(V_j, \alpha_j), \quad j = 1, \ldots, N_t. \tag{6}$$

This model is consistent with Lewandowski [2004] where the lift force is proportional to $V_j^2 \alpha_j$ and the drag force is proportional to $V_j^2 \alpha_j^2$ for small angles of attack. To account for dynamics in the thrusters, the derivatives of the thruster angles and velocities are considered as control inputs to the system. A compact notation of the state vector is thus defined as $\boldsymbol{x} = \begin{bmatrix} \boldsymbol{\eta}^\mathsf{T} & \boldsymbol{\nu}^\mathsf{T} & \boldsymbol{\alpha}^\mathsf{T} & \boldsymbol{n}^\mathsf{T} \end{bmatrix}^\mathsf{T}$ and the control-input vector is $\boldsymbol{u} = \begin{bmatrix} \dot{\boldsymbol{\alpha}}^\mathsf{T} & \dot{\boldsymbol{n}}^\mathsf{T} \end{bmatrix}^\mathsf{T}$, where $\boldsymbol{\alpha} = \begin{bmatrix} \alpha_1 & \ldots & \alpha_{N_t} \end{bmatrix}^\mathsf{T}$ and $\boldsymbol{n} = \begin{bmatrix} n_1 & \ldots & n_{N_t} \end{bmatrix}^\mathsf{T}$. Then by combining (1)-(6), the ship dynamics can be written as:

$$\dot{\boldsymbol{x}}(t) = \boldsymbol{f}_{\text{dyn}}(\boldsymbol{x}(t), \boldsymbol{u}(t)). \tag{7}$$

The feasible set for the state space is given by

$$\mathcal{X} = \{\boldsymbol{x} \mid ||\begin{bmatrix} u & v \end{bmatrix}||_2 \leq v_{\max},\ |n_j| \leq \bar{n}, \quad j = 1, \ldots, N_t\}, \tag{8}$$

where $v_{\max}$ is the maximum allowed magnitude of the ship's velocity and $\bar{n}$ is the maximum speed for the thrusters. The control inputs are restricted to

$$\mathcal{U} = \{\boldsymbol{u} \mid |\dot{\alpha}_j| \leq \bar{\dot{\alpha}}_j, \quad |\dot{n}_j| \leq \bar{\dot{n}}_j, \quad j = 1, \ldots, N_t\}. \tag{9}$$

The ship is assumed to operate in an environment with only static obstacles $\mathcal{X}_{\text{obst}}$. The free-space where the vehicle is not in collision with any obstacle is defined as $\mathcal{X}_{\text{free}} = \mathcal{X} \setminus \mathcal{X}_{\text{obst}}$. Since the free-space $\mathcal{X}_{\text{free}}$ is defined as the complement set of $\mathcal{X}_{\text{obst}}$, it is in general non-convex.

The motion planning problem is defined as follows: compute a feasible and collision-free state and control input trajectory $(\boldsymbol{x}(t), \boldsymbol{u}(t))$, $t \in [0, T_f]$ that moves the ship from its initial state $\boldsymbol{x}(0) = \boldsymbol{x}_{\text{init}} \in \mathcal{X}_{\text{free}}$ to a desired goal state $\boldsymbol{x}(T_f) = \boldsymbol{x}_{\text{term}} \in \mathcal{X}_{\text{free}}$, while minimizing an objective functional J. In this work, the cost function used to define the objective functional is separated into two parts, where $\ell(\boldsymbol{x}, \boldsymbol{u})$ is the first part that is position invariant, which can, e.g., be related to time and energy consumption. The second part is denoted ℓ_{env} and depends on the generalized position of the ship. It can, e.g., represent costs related to safety. In this work, the cost ℓ_{env} is related to the distance to obstacles:

$$\ell_{\text{env}}(d) = \begin{cases} k_d (d_{\text{safe}} - d)^2, & 0 < d \leq d_{\text{safe}} \\ 0, & d > d_{\text{safe}} \end{cases}, \tag{10}$$

where d is the distance from the bounding region of the ship's body to the closest obstacle in $\mathcal{X}_{\text{obst}}$, d_{safe} is a (soft) desired safety distance and $k_d > 0$ is a weighting parameter. The distance d is formally defined as $d = \text{distance}(\mathcal{S}_b(\boldsymbol{x}), \mathcal{X}_{\text{obst}})$, where the bounding region of the ship's body $\mathcal{S}_b(\boldsymbol{x})$ is represented as a convex polytope and depends of the state $\boldsymbol{x}$, see Fig. 1. Now, it is possible to pose the motion planning problem as the following continuous-time OCP:

$$\begin{aligned} \underset{\boldsymbol{u}(\cdot), T_f}{\text{minimize}} \quad & J = \int_0^{T_f} [\ell(\boldsymbol{x}(t), \boldsymbol{u}(t)) + \ell_{\text{env}}(d(t))]\, \mathrm{d}t \\ \text{subject to} \quad & \boldsymbol{x}(0) = \boldsymbol{x}_{\text{init}}, \quad \boldsymbol{x}(T_f) = \boldsymbol{x}_{\text{term}}, \\ & \dot{\boldsymbol{x}}(t) = \boldsymbol{f}_{\text{dyn}}(\boldsymbol{x}(t), \boldsymbol{u}(t)), \\ & d(t) = \text{distance}(\mathcal{S}_b(\boldsymbol{x}(t)), \mathcal{X}_{\text{obst}}), \\ & \boldsymbol{x}(t) \in \mathcal{X}_{\text{free}},\ \boldsymbol{u}(t) \in \mathcal{U}. \end{aligned} \tag{11}$$

The upcoming sections describe how the proposed framework for solving (11) is implemented by using a combination of a lattice-based motion planner and an optimization-based improvement step.

3 Lattice-based motion planner

A lattice-based motion planner computes a suboptimal solution to the continuous OCP in (11) by transforming it as a graph-search problem. This transformation is done by restricting the controls to a finite subset of the available actions. In this work, the so-called state-lattice methodology is used Pivtoraiko et al. [2009], where the set of available actions is represented using a motion-primitive set $\mathcal{P}$. The construction of this set is performed offline and can be divided into two main steps: state-space discretization and motion-primitive computation.

3.1 State-space discretization

Before the motion primitive set can be computed, a discrete representation of the ship's state space $\mathcal{X}_d$ needs to be selected. Since the complexity of the online graph-search problem grows exponentially with the dimension of the search space LaValle [2006], it is important to carefully select the discretization $\mathcal{X}_d$. In this work, the position (x, y) of the ship

is discretized onto a uniform grid with resolution r_p. The orientation $\psi \in \Psi$, $|\Psi| = 16$, is irregularly discretized as proposed in Pivtoraiko et al. [2009] to enable the computation of short sway and surge motion primitives. The velocity $\boldsymbol{\nu}_d \in \mathbf{V}$ is also discretized such that short sway and surge maneuvers can be performed, with rotational velocity $r_d = 0$ for all $\boldsymbol{\nu}_d$. One such discretization, that will be used in Section 5, is to include surge maneuvers with $u_d \in \{0, \pm\frac{v_{\max}}{2}, v_{\max}\}$ and sway maneuvers with $v_d \in \{0, \pm\frac{v_{\max}}{2}\}$.

To be able to compute short sway or surge motion primitives for the discretized values of $\boldsymbol{\nu}_d \in \mathbf{V}$, the corresponding discretized values $\boldsymbol{\alpha}_d$ and $\boldsymbol{n}_d$ are selected such that the ship obtains a configuration where $\dot{\boldsymbol{\nu}}_d = 0$. From the dynamics of the ship given in (3), this configuration can be obtained by selecting $\boldsymbol{\tau}_d$ such that:

$$\boldsymbol{M}\dot{\boldsymbol{\nu}}_d = 0 \iff \boldsymbol{\tau}_d = -\boldsymbol{C}(\boldsymbol{\nu}_d) - \boldsymbol{D}(\boldsymbol{\nu}_d). \tag{12}$$

Finally, to obtain the thruster angles and velocities $\boldsymbol{\alpha}_d$ and $\boldsymbol{n}_d$ that generates the desired $\boldsymbol{\tau}_d$ from (12), the following feasibility problem obtained from (4) is solved:

$$\begin{aligned} &\text{find} \quad \boldsymbol{\alpha}_d, \boldsymbol{n}_d \\ &\text{s.t.} \quad \boldsymbol{\tau}_d = \sum_{i=j}^{M} \boldsymbol{T}\left(\theta_{p,j}\boldsymbol{\tau}_p(\alpha_{j,d}, n_{j,d}) + \theta_{r,j}\boldsymbol{\tau}_r(\alpha_{j,d}, \boldsymbol{\nu}_d)\right), \end{aligned} \tag{13}$$

which gives the complete state-space discretization $\mathcal{X}_d$. However, it should be stressed that on the trajectory between two discretized states, the ship can take any feasible state in $\mathcal{X}$.

3.2 Motion primitive computation

A motion primitive $\boldsymbol{m}$ is defined as a feasible state and control-input trajectory that moves the ship from an initial state $\boldsymbol{x}_0 \in \mathcal{X}_d$ to a terminal state $\boldsymbol{x}_f \in \mathcal{X}_d$ in time T, i.e.,

$$\boldsymbol{m} = \big(\boldsymbol{x}(t), \boldsymbol{u}(t)\big) \in \mathcal{X} \times \mathcal{U},\ t \in [0, T]. \tag{14}$$

To compute the motion primitives $\boldsymbol{m} \in \mathcal{P}$, the optimization-based framework in Bergman et al. [June 2019] is used. Since the position-dependent cost (ℓ_{env} in (11)) is unknown offline, the motion primitives are computed by solving OCPs where only the integral of the position invariant cost $\ell_m(\boldsymbol{m}) = \int_0^T \ell(\boldsymbol{x}, \boldsymbol{u})\mathrm{d}t$ is minimized. In this work, the first type of maneuvers are short surge and sway trajectories where the velocity is kept constant. The second typ of maneuvers are trajectories to neighboring headings and velocities in $\mathcal{X}_d$. From each initial state, trajectories to the closest neighbors in $\mathbf{V}$ combined with heading changes to the eight closest neighbors in Ψ are computed. To reduce the branching factor of the online graph-search problem, heading changes are limited to the closest adjacent headings from initial states with $u_d = v_{\max}$. Moreover, since the ship is orientation invariant, rotational symmetries Pivtoraiko and Kelly [2005] are exploited to reduce the number of OCPs needed to be solved when $\mathcal{P}$ is computed (illustrated in Fig. 3). Together, these maneuvers define the motion primitive set $\mathcal{P}$. Since the ship is position invariant, each $\boldsymbol{m} \in \mathcal{P}$ can be translated and reused from each discrete position in $\mathcal{X}_d$.

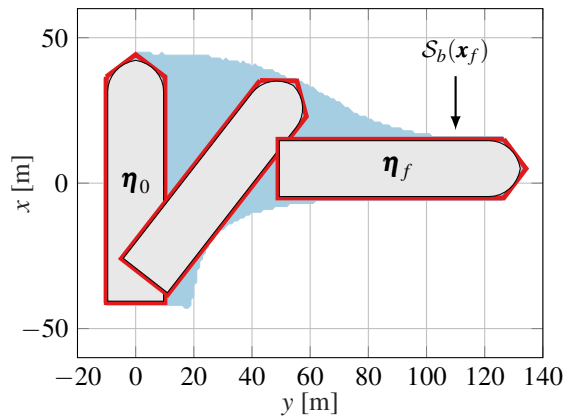


Figure 2: The swath (blue area) of a motion primitive that moves the ship from $\boldsymbol{\eta}_0 = [0, 0, 0]^\intercal$, $\boldsymbol{\nu}_0 = [0, 1.5433\text{m/s}, 0]^\intercal$, to $\boldsymbol{\eta}_f = [5\text{m}, 90\text{m}, \pi/2]^\intercal$, $\boldsymbol{\nu}_f = [1.5433\text{m/s}, 0, 0]^\intercal$.

3.3 Collision avoidance and environmental cost

The environment used within the lattice-based motion planner is selected to be represented using a cost map Ferguson and Likachev [2008]. The cost map is both used to ensure collision avoidance and to obtain an approximation of the position-dependent cost ℓ_{env} in (11). For efficient online collision checking, we precompute *swaths* Pivtoraiko et al. [2009] for each motion primitive in $\mathcal{P}$. A swath is represented as the set of cells that the volume of the ship is occupying during an execution of a motion primitive. In this work, a convex polytope $\mathcal{S}_b$ is used to represent the volume of the ship as shown in Fig. 1. One example of a swath for a right-turn maneuver is illustrated in Fig. 2. Furthermore, it is also possible to approximate the position-dependent cost for $\boldsymbol{m}$ applied from $\boldsymbol{x}$ as

$$\ell_{m,\text{env}}(\boldsymbol{x}, \boldsymbol{m}, \mathcal{X}_{\text{obst}}) = \int_0^T \ell_{\text{env}}(d(t))\mathrm{d}t, \tag{15}$$

where $d(t) = \text{distance}(\mathcal{S}_b(\boldsymbol{x}, \boldsymbol{m}(t)), \mathcal{X}_{\text{obst}})$. Since only static obstacles are considered in this work, it is possible to precompute an approximation of this function. This approximation is then used as the cost map during planning.

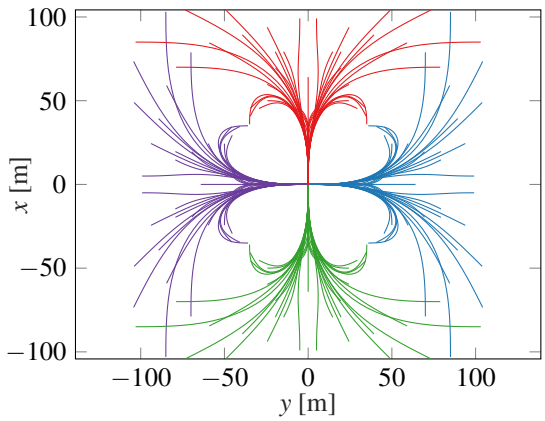


Figure 3: Illustration of the paths for motion primitives from four different initial headings, $\psi_0 = \frac{k\pi}{2}$, $k = 0,1,2,3$ and $\boldsymbol{v}_0 = [1.5433\text{m/s}, 0, 0]^\intercal$ to neighboring states in $\mathcal{X}_d$.

3.4 Resulting graph-search problem

Now, the resulting graph-search problem can be posed as a discrete OCP in the form:

$$\underset{\{m_k\}_{k=0}^{M-1},\, M}{\text{minimize}} \quad J_m = \sum_{k=0}^{M-1} \ell_m(\boldsymbol{m}_k) + \ell_{m,\text{env}}(\boldsymbol{x}_k, \boldsymbol{m}_k, \mathcal{X}_{\text{obst}}) \tag{16a}$$

$$\text{subject to} \quad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}}, \quad \boldsymbol{x}_M = \boldsymbol{x}_{\text{term}}, \tag{16b}$$

$$\boldsymbol{x}_{k+1} = \boldsymbol{f}_m(\boldsymbol{x}_k, \boldsymbol{m}_k), \tag{16c}$$

$$\boldsymbol{m}_k \in \mathcal{P}(\boldsymbol{x}_k), \tag{16d}$$

$$c(\boldsymbol{x}_k, \boldsymbol{m}_k) \notin \mathcal{X}_{\text{obst}}, \tag{16e}$$

where the decision variables are the motion-primitive sequence $\{m_k\}_{k=0}^{M-1}$ and its length M. The state transition equation in (16c) outputs the successor state $\boldsymbol{x}_{k+1}$ when $\boldsymbol{m}_k$ is applied from $\boldsymbol{x}_k$, and (16d) restricts the choice of $\boldsymbol{m}_k$ to the set of applicable motion primitives $\mathcal{P}(\boldsymbol{x}_k)$ from state $\boldsymbol{x}_k$. Finally, the constraint in (16e) ensures that the ship does not collide with obstacles when $\boldsymbol{m}_k$ is applied from $\boldsymbol{x}_k$.

The problem in (16) can be solved using graph-search algorithms Pivtoraiko et al. [2009]. For a faster search, a precomputed free-space heuristic look-up table (HLUT) Knepper and Kelly [2006] is used as heuristic function. It is computed by exploiting rotational symmetries and solving (16) in free-space to all neighboring states within a predefined distance. Similar to the computation of $\mathcal{P}$, rotational symmetries are exploited to reduce the size of the HLUT. The output from solving (16) is hereafter referred to as the *nominal trajectory* $(\bar{\boldsymbol{x}}(\cdot), \bar{\boldsymbol{u}}(\cdot))$ which is used to warm-start the optimization-based improvement step.

4 Optimization-based improvement

In this section, it is shown how the nominal trajectory computed by the lattice-based planner can be used as warm-start to a receding-horizon optimization-based improvement step. The aim is to remove the suboptimality introduced when solving (16) and to efficiently find a locally optimal solution to the original motion planning problem in (11).

4.1 Collision avoidance and environmental cost

One important aspect for reducing the complexity of the optimization-based improvement step is to use an efficient representation of the available free space. One approach is to use individual bounding regions of each obstacle LaValle [2006], but the complexity using this approach scales poorly when the number of obstacles increases. Instead, a more computationally efficient approach is used where so-called safety envelopes Liu et al. [2017], Schoels et al. are computed, which represent local inner-approximations of the ship's obstacle-avoidance constraints in (11). However, previous methods for computing these safety envelopes have only shown robustness for vehicles that can be described by a single bounding circle, i.e., not for ships that is considered in this work.

By using direct methods for optimal control, the improvement OCP is constructed by discretizing the continuous-time motion planning problem (11), e.g., using direct multiple-shooting with $N+1$ discretization points Diehl et al. [2006]. At each discrete point i, $i = 0, \ldots, N$, a local spatial constraint will be computed for collision avoidance that is represented by a convex polytope $\mathcal{S}^i_{\text{env}}$ which is defined as

$$\mathcal{S}^i_{\text{env}} = \{\boldsymbol{p} \in \mathbf{R}^2 \mid \boldsymbol{A}_i \boldsymbol{p} \leq \boldsymbol{b}_i\}, \tag{17}$$

where $\boldsymbol{A}_i = [\boldsymbol{a}_{i,1} \ \ldots \ \boldsymbol{a}_{i,K_i}]^\intercal \in \mathbf{R}^{K_i \times 2}$, $\boldsymbol{a}_{i,j} \in \mathbf{R}^2$, $||\boldsymbol{a}_{i,j}||_2 = 1$, for $j = 1, \ldots, K_i$, $\boldsymbol{b}_i \in \mathbf{R}^{K_i}$ and K_i is the number of half spaces that defines $\mathcal{S}^i_{\text{env}}$. These polytopes are computed using Algorithm 1, where each polytope $\mathcal{S}^i_{\text{env}}$ is temporarily transformed to a directed cyclic graph $\mathcal{G}^i_{\text{env}} = < \mathcal{V}^i_{\text{env}}, \mathcal{E}^i_{\text{env}} >$, where the vertices $\boldsymbol{p}_k \in \mathcal{V}^i_{\text{env}}$ define the convex hull of $\mathcal{S}^i_{\text{env}}$ and each edge $e_k \in \mathcal{E}^i_{\text{env}}$ represents a vector from $\boldsymbol{p}_k$ to $\boldsymbol{p}_{k+1}$. Each vertex $\boldsymbol{p}_k$ is associated with an expansion direction $\bar{\boldsymbol{g}}_k$ which is selected such that the area spanned by the graph increases after each expansion. One such example is to select the directions $\bar{\boldsymbol{g}}_k$ to be aligned with the vectors pointing from the center of the ship's body to each of the initial vertices.

The inputs to the algorithm are the nominal state from the lattice-based planner at point i, i.e., $\boldsymbol{\eta}_i$ and $\boldsymbol{v}_i$, the polytope $\mathcal{S}_b$ representing the ship's body and the edge set $\mathcal{E}_{\text{obst}}$ that represents surrounding obstacles. The graph is initialized using $\mathcal{S}_b$ at $\boldsymbol{\eta}_i$ (Line 2). Then, the initial candidate vertex $\boldsymbol{p}_c$ is selected as the one whose expansion direction $\bar{\boldsymbol{g}}$, see Fig. 4, is closest to the ship's velocity vector $\boldsymbol{v}_i$. The expansion of the graph $\mathcal{G}^i_{\text{env}}$ continues as long as at least one vertex is expandable. A vertex is no longer considered expandable if its corresponding step length Δ_l is below a minimum value or if a maximum expansion distance is reached. If a candidate vertex is expandable, it is expanded in its expansion direction $\bar{\boldsymbol{g}}$ using step size Δ_l (Line 8).

After a vertex expansion, it is first checked that the area spanned by the graph is convex. If this is not the case, the algorithm will consume the limiting vertex if the cardinality of

$\mathcal{V}^i_{\text{env}}$ is above a certain threshold (Line 11), and the reduced vertex set does not intersect any obstacle (Line 12). Otherwise, the step is rejected, Δ_l is halved and the expansion continues from its child (Line 17). The next step is to calculate the number of intersections n_{lines} between $\mathcal{E}^i_{\text{env}}$ and $\mathcal{E}_{\text{obst}}$ (Line 20). Depending on the value of n_{lines}, three cases can occur:

1. *Exactly one edge in $\mathcal{E}_{\text{obst}}$ intersected* (Line 21). The current vertex is first projected to feasibility by reducing the step size such that $\boldsymbol{p}_c$ lies on the intersected edge. Then, it is duplicated and inserted in the set of vertices. The expansion directions for $\boldsymbol{p}_c$ and $\boldsymbol{p}_{c,\text{new}}$ are modified such that they grow in each direction parallel to the edge that was intersected.

2. *More than one edge in $\mathcal{E}_{\text{obst}}$ intersected* (Line 26). The step is rejected and the step size Δ_l is halved.

3. Otherwise, the step is accepted and the expansion continues with the next vertex in $\mathcal{V}^i_{\text{env}}$ (Line 31).

Fig. 4 illustrates how Algorithm 1 computes a local spatial polytope $\mathcal{S}^i_{\text{env}}$ in an area with obstacles using the nominal trajectory computed by the lattice-based planner.

The output from Algorithm 1 is a convex polytope $\mathcal{S}^i_{\text{env}}$ that is computed for each discrete point i of the improvement step. Since both $\mathcal{S}_b(\boldsymbol{x}_i)$ and $\mathcal{S}^i_{\text{env}}$ are convex, a sufficient condition to ensure collision avoidance for the ship's body is that all vertices $\mathcal{V}_b$ of $\mathcal{S}_b(\boldsymbol{x}_i)$ lie within $\mathcal{S}^i_{\text{env}}$ Martinsen et al. [2019]. This condition can be described by the following constraint:

$$\boldsymbol{A}_i \boldsymbol{c}_{\text{rot}}(\boldsymbol{x}_i, \boldsymbol{p}_j) \leq \boldsymbol{b}_i, \quad \forall \boldsymbol{p}_j \in \mathcal{V}_b, \tag{18}$$

where

$$\boldsymbol{c}_{\text{rot}}(\boldsymbol{x}_i, \boldsymbol{p}_j) = \begin{bmatrix} \cos\psi_i & -\sin\psi_i \\ \sin\psi_i & \cos\psi_i \end{bmatrix} \boldsymbol{p}_j + \begin{bmatrix} x_i \\ y_i \end{bmatrix}, \tag{19}$$

and $\boldsymbol{A}_i$ and $\boldsymbol{b}_i$ are the half-space representation of $\mathcal{S}^i_{\text{env}}$ defined in (17). Furthermore, we want to express the position-dependent cost related to the distance to obstacles also in the improvement step. As a consequence, the following modification of (18) is introduced:

$$\boldsymbol{A}_i \boldsymbol{c}_{\text{rot}}(\boldsymbol{x}_i, \boldsymbol{p}_j) \leq \boldsymbol{b}_i - \mathbf{1}_i (d_{\text{safe}} - \varepsilon_{d,i}), \quad \forall \boldsymbol{p}_j \in \mathcal{V}_b, \tag{20}$$

where a variable $\varepsilon_{d,i}$ which satisfies $0 \leq \varepsilon_{d,i} \leq d_{\text{safe}}$ is added, which is used to represent the distance to the boundaries of $\mathcal{S}^i_{\text{env}}$. This modification makes it possible to compute an approximation of ℓ_{env} in (11) since the distance to obstacles is $d_i \geq d_{\text{safe}} - \varepsilon_{d,i}$ when (20) holds. Note that the feasible sets for (18) and (20) are equal, as (18) is obtained from (20) by selecting $\varepsilon_{d,i} = d_{\text{safe}}$.

Algorithm 1 Computation of local spatial constraints

1: **Input**: $\bar{\boldsymbol{\eta}}_i$, $\bar{\boldsymbol{v}}_i$, $\mathcal{S}_b$, $\mathcal{E}_{\text{obst}}$.
2: $< \mathcal{V}^i_{\text{env}}, \mathcal{E}^i_{\text{env}} >$ = initialize_graph($\mathcal{S}_b, \bar{\boldsymbol{\eta}}_i$)
3: $\boldsymbol{p}_c \leftarrow$ select_initial($\mathcal{V}^i_{\text{env}}$, $\bar{\boldsymbol{v}}_i$)
4: **while** any($\boldsymbol{p}_k$.expandable(), $\forall \boldsymbol{p}_k \in \mathcal{V}_{\text{env}}$) **do**
5: **if** $\boldsymbol{p}_c$.expandable() **then**
6: $\Delta_l \leftarrow \boldsymbol{p}_c$.get_step_length()
7: $\bar{\boldsymbol{g}} \leftarrow \boldsymbol{p}_c$.get_direction()
8: $\boldsymbol{p}_c \leftarrow \boldsymbol{p}_c + \Delta_l \bar{\boldsymbol{g}}$
9: **if** span($\mathcal{V}^i_{\text{env}}$) **not** convex **then**
10: success = **False**
11: **if** $|\mathcal{V}^i_{\text{env}}| > n_{\text{card}}$ **then**
12: success $\leftarrow \mathcal{V}^i_{\text{env}}$.remove_vertex()
13: **end if**
14: **if not** success **then**
15: $\boldsymbol{p}_c \leftarrow \boldsymbol{p}_c - \Delta_l \bar{\boldsymbol{g}}$
16: $\boldsymbol{p}_c$.set_step_length($\Delta_l/2$)
17: **continue**
18: **end if**
19: **end if**
20: $n_{\text{lines}} \leftarrow$ check_intersections($\mathcal{E}^i_{\text{env}}$, $\mathcal{E}_{\text{obst}}$)
21: **if** $n_{\text{lines}} = 1$ **then**
22: $\boldsymbol{p}_c \leftarrow$ project_to_feasibility($\boldsymbol{p}_c$)
23: $\boldsymbol{p}_{c,\text{new}} \leftarrow \boldsymbol{p}_c$
24: update_directions($\boldsymbol{p}_{c,\text{new}}$, $\boldsymbol{p}_c$)
25: $\mathcal{V}_{\text{env}}$.insert_vertex($\boldsymbol{p}_{c,\text{new}}$)
26: **else if** $n_{\text{lines}} > 1$ **then**
27: $\boldsymbol{p}_c \leftarrow \boldsymbol{p}_c - \Delta_l \bar{\boldsymbol{g}}$
28: $\boldsymbol{p}_c$.set_step_length($\Delta_l/2$)
29: **end if**
30: **end if**
31: $\boldsymbol{p}_c$ = $\boldsymbol{p}_c$.get_child()
32: **end while**

4.2 Improvement optimization problem

The improvement step can now be posed as the following discrete OCP at the current state of the ship defined by $\boldsymbol{x}_{\text{cur}}$:

$$\begin{aligned} \underset{\{\boldsymbol{u}_i, \varepsilon_{d,i}\}_{i=0}^{N-1},\, \Delta_t}{\text{minimize}} \quad & J_d = \sum_{i=0}^{N-1} \ell_d(\boldsymbol{x}_i, \boldsymbol{u}_i, \Delta_t) + \ell_{d,\text{env}}(d_{\text{safe}} - \varepsilon_{d,i}, \Delta_t) \\ \text{subject to} \quad & \boldsymbol{x}_0 = \boldsymbol{x}_{\text{cur}}, \quad \boldsymbol{x}_N = \bar{\boldsymbol{x}}(\xi), \\ & \boldsymbol{x}_{i+1} = \boldsymbol{f}_d(\boldsymbol{x}_i, \boldsymbol{u}_i, \Delta_t), \\ & \boldsymbol{A}_i \boldsymbol{c}_{\text{rot}}(\boldsymbol{x}_i, \boldsymbol{p}_j) \leq \boldsymbol{b}_i - \mathbf{1}_i (d_{\text{safe}} - \varepsilon_{d,i}), \ \forall \boldsymbol{p}_j \in \mathcal{V}_b \\ & \boldsymbol{x}_i \in \mathcal{X} \quad \boldsymbol{u}_i \in \mathcal{U}, \quad 0 \leq \varepsilon_{d,i} \leq d_{\text{safe}}. \end{aligned} \tag{21}$$

Here, Δ_t is a decision variable that represents the time between two consecutive discrete points. The objective function J_d is the sum of the numerical integrals of ℓ and ℓ_{env} in (11) over the time interval Δ_t. Furthermore, the parameter ξ is used to decide where to connect to the nominal trajectory. Hence, it is possible to solve the problem once with $\boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}}$ and $\boldsymbol{x}_N = \bar{\boldsymbol{x}}(T_f) = \boldsymbol{x}_{\text{term}}$, or repeatedly using a receding-horizon approach with ξ and $\boldsymbol{x}_{\text{cur}}$ updated at each iteration to maintain a desired planning horizon $T = N\Delta_t$ Bergman et al.

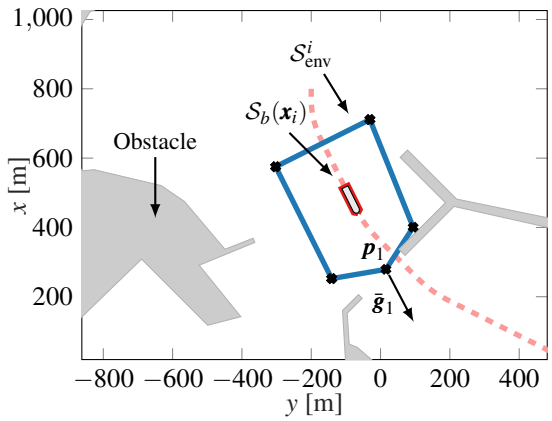


Figure 4: Illustration of one resulting convex bounding box ($\mathcal{S}^i_{\text{env}}$) computed from a state on the nominal trajectory (dashed). Collision avoidance is ensured by keeping the vertices of the ship $\mathcal{S}_b(\boldsymbol{x}_i)$ (red) within the local spatial constraints $\mathcal{S}^i_{\text{env}}$ (blue).

[b]. In both cases, the nominal trajectory is used both as warm-start, but also to compute the local convex spatial constraints using Algorithm 1. Since the initial nominal trajectory is feasible, the improvement step is guaranteed to find a solution which is at least as good as the initial one Bergman et al. [b]. As previously mentioned, the spatial constraints in (21) are only *local* approximations of the free space computed from the nominal trajectory. To avoid unnecessary computational burden when a receding-horizon approach is used, these spatial constraints are only re-computed if any of the constraints in (18) are close to become active. Otherwise, the constraints from the previous iteration are reused in the next iteration.

Finally, note that the proposed improvement step can be used to transform the optimal open-loop control law computed by the lattice-based planner to a optimal closed-loop control law if it is repeated at a sufficiently high rate. This will make it possible to suppress various disturbances acting on the ship and changes in the environment during trajectory execution. Another possibility is to let changes in the environment trigger replanning and use a trajectory-tracking controller such as Bärlund et al. to stabilize the ship around the more or less static trajectory computed by the improvement step.

5 Simulation study

In this section, the proposed framework is evaluated in a simulation study by solving motion planning problems in a model of the Cape Town harbor. The geometry of the harbor is represented using a polygon that has been calculated using data from Google maps. The cost map used in the lattice-based planner is computed with resolution 1 m. The lattice-based planner is implemented in C++ and the improvement step in Python using CasADi Andersson et al. [In Press, 2018] with IPOPT Wächter and Biegler [2006] as NLP solver, and discretized using multiple shooting.

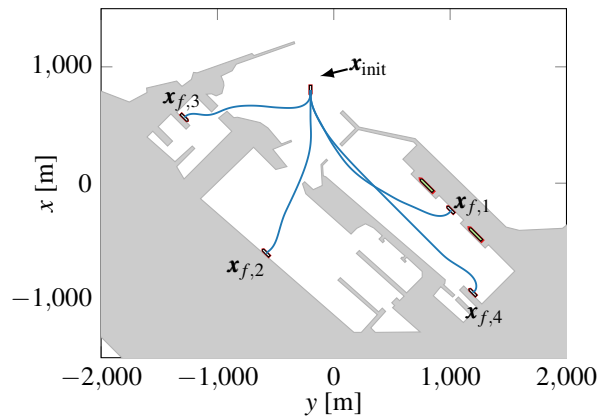


Figure 5: Simulation-study scenario with resulting paths (blue). Planning from an initial state on the fairway to four terminal docking states $\boldsymbol{x}_{f,j}$, $j = 1,\dots,4$ in the harbor.

The simulations are based on a model of a small ship with two thrusters. The model of the ship is based on the *supply vessel* model in the MSS hydro toolbox Perez et al. [2006]. The parameters for the two thrusters are:

$$\begin{aligned} &l_x = \pm 32\,\text{m}, \quad l_y = 0, \quad \bar{n} = 2\,\text{RPS}, \\ &\bar{\dot{n}} = 0.08\,\text{RPS/s}, \quad \bar{\dot{\alpha}} = 7.2^\circ/\text{s}, \end{aligned} \tag{22}$$

and the propeller and rudder gains are $\theta_p = 4.2 \cdot 10^5$ and $\theta_r = 3.8 \cdot 10^4$, respectively. Hence, one thruster is mounted in the stern, as shown in Fig. 1, and the other in the bow. The lattice-based planner is discretized with a position resolution of $r_p = 5$ m, and the maximum velocity $v_{\max}$ is set to 6 kn. The position-invariant cost function used in both steps of the framework is selected as

$$\ell(\boldsymbol{x},\boldsymbol{u}) = 1 + 0.1\boldsymbol{n}^\mathsf{T}\boldsymbol{n} + 100\dot{\boldsymbol{n}}^\mathsf{T}\dot{\boldsymbol{n}} + 100\dot{\boldsymbol{\alpha}}^\mathsf{T}\dot{\boldsymbol{\alpha}}, \tag{23}$$

to trade-off between time, energy and smoothness. For the position-dependent cost ℓ_{env}, the value of weighting parameter is selected as $k_d = 1.5 \cdot 10^{-3}$ in (10) and (21), and the safety distance parameter is set to $d_{\text{safe}} = 20$m which is approximately equal to the width of the ship's body.

The motion-planning scenario is illustrated in Fig. 5, where four different trajectories are computed from an initial state on the fairway. The terminal states $\boldsymbol{x}_{f,j}$ represent docking states with zero velocity. Since the performance of the proposed motion planner is evaluated in this section, the simulations are performed without any disturbances and the open-loop control law is used for evaluation.

First, the impact of using different planning horizons in the improvement step is analyzed. The desired sampling time is selected as $\Delta_t = 2$ s in (21). Hence, the number of discretization points N defines the desired planning horizon $T = \Delta_t N$. A suitable value of T is both system and problem specific. Here, the value of T is selected by evaluating several planning horizons applied on the scenarios shown in Fig. 5. The results are summarized in Fig. 6, which presents the reduction in objective function value compared to the nominal trajectory as a function of the planning horizon T. Furthermore, the

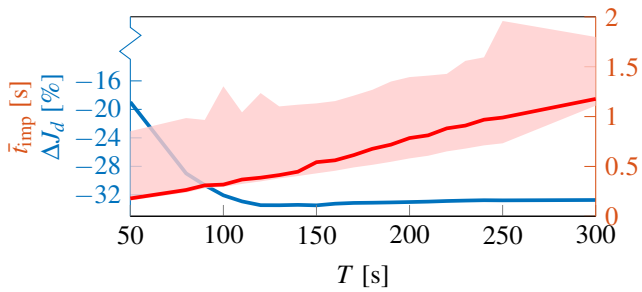


Figure 6: Results from the improvement step when planning a motion to $\boldsymbol{x}_{f,1}$ for different planning horizons T. Blue: improvement in objective function value compared to the nominal trajectory. Red: median computation time for the improvement step. The shaded area shows the span of computation times below the 95th percentile.

computation times for the corresponding values of T are also displayed. The results indicate that after a planning horizon of 150 s, no further improvement in objective function value is obtained, while the computation time continues to increase. Therefore, $T = 150$ s is a reasonable choice and is used in the remainder of this section. Moreover, with $T = 150$ s, the 95th percentile of the computation time is 1.13 s (median: 0.7 s), which is well below the desired sampling time $\Delta_t = 2$ s.

Table 1 highlights the benefits of improving the nominal trajectory. Both the time T_f to reach the terminal state and the amount of force applied by the thrusters F_{tot} are reduced. In particular, the applied force is significantly reduced by 40–50% in the given scenarios. The enhancements come at the expense of a minor latency time, arising from the computation time of the initial spatial constraints t_{env} and the first receding-horizon improvement step. These minor initial computations could, however, be performed already when the ship is approaching the initial state at which the plan starts. In summary, smoother and more energy-efficient solutions are in these scenarios obtained using the receding-horizon improvement step. These results are illustrated for one scenario in Fig. 7–8, where the trajectory computed by the improvement step is compared to the nominal trajectory computed by the lattice-based planner.

Finally, Fig. 9 illustrates the impact of adding the safety-distance parameter to (21). When the distance to obstacles is not penalized, i.e., $k_d = 0$, it can be seen that the ship is very close to the boundary of an obstacle ($d_{\min} = 0.2$ m). In contrast, when $k_d = \bar{k}_d = 1.5 \cdot 10^{-3}$, the desired safety distance d_{safe} is only violated during the final phase of the maneuver ($t > 370$ s) mainly due to the location of the terminal state. Note that the terminal state is reached *faster* when $k_d > 0$. This is because the environmental cost is integrated over time, which means that the time spent with $d(t) < d_{\text{safe}}$ is minimized. One possibility to alleviate this behavior is to choose the value of k_d adaptively based on, e.g., the current distance to the terminal state.

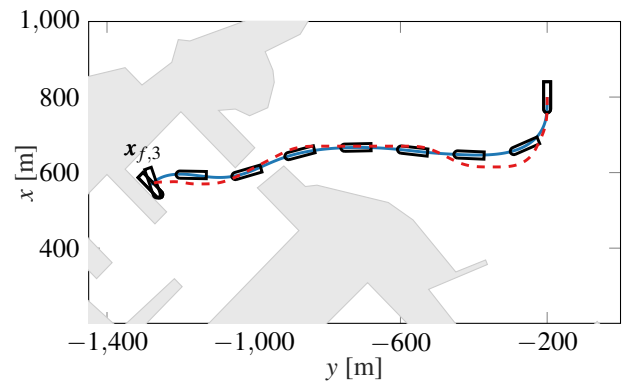
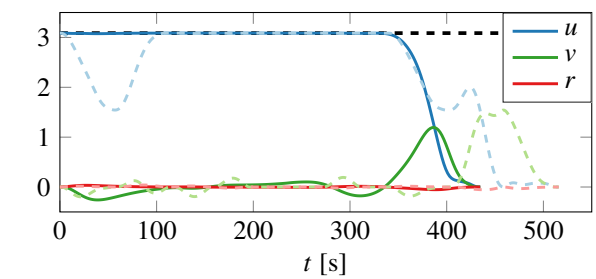
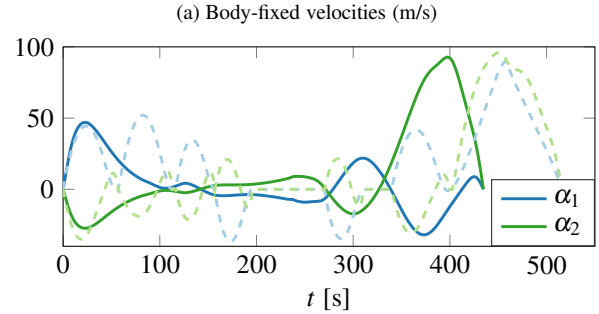


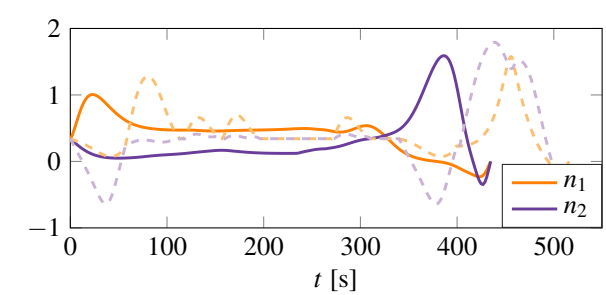
Figure 7: The nominal trajectory (dotted red) and the solution from the improvement step (blue) when planning a motion to $\boldsymbol{x}_{f,3}$.



(a) Body-fixed velocities (m/s)



(b) Thruster angles (°)



(c) Thruster velocities (RPS)

Figure 8: Trajectories for the problem illustrated in Fig. 7 using a planning horizon $T = 150$ s. The dashed lines represent the nominal trajectory from the lattice-based planner.

Table 1: Results from the four docking problems in Fig. 5 using a planning horizon $T = 150$ s. t_{lat} and t_{env} are the times to compute the nominal trajectory and the initial spatial envelopes, respectively. $\bar{t}_{\text{imp}}$ is the median computation time for the improvement step. Finally T_f and F_{tot} are the terminal time and total applied force, respectively, for the nominal trajectory (lat) and after the improvement step (imp).

$\boldsymbol{x}_{f,j}$	t_{lat} [s]	t_{env} [s]	$\bar{t}_{\text{imp}}$ [s]	$T_{f,\text{lat}}$ [s]	$T_{f,\text{imp}}$ [s]	$F_{\text{tot,lat}}$ [MN]	$F_{\text{tot,imp}}$ [MN]
1	3.0	1.4	0.46	652	617	152	90
2	0.19	1.3	0.55	563	532	131	67
3	2.3	1.1	0.54	510	435	155	86
4	6.6	1.1	0.44	826	807	206	120

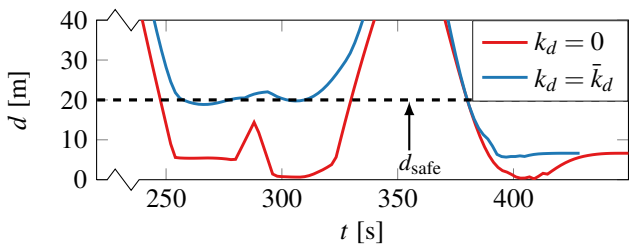


Figure 9: Distance to obstacles along the trajectory for the problem illustrated in Fig. 7. The blue line is obtained when using a value of $k_d = \bar{k}_d = 1.5 \cdot 10^{-3}$, and the red line when $k_d = 0$. The black dashed line represents the desired safety distance d_{safe}.

6 Conclusions and Future Work

This paper introduces a two-step optimization-based motion planner for maneuvering of ships in confined environments such as harbors. In the first step, a lattice-based motion planner solves the combinatorial aspects of the problem and computes a feasible, but suboptimal solution to the problem using a finite library of optimized maneuvers. In the second step, the solution from the lattice-based planner is improved using receding-horizon optimization techniques. For efficient obstacle avoidance, a novel algorithm for automatic computation of safe spatial envelopes is introduced. Furthermore, a safety-distance parameter is introduced in the optimization-based improvement step which enables the algorithm to maintain a desired safety distance to obstacles without reducing the feasible region. The proposed motion planner is applied to a set of autonomous docking problems in a model of the Cape Town harbor, where it successfully computes energy-efficient solutions even in narrow passages.

Future work includes to extend the algorithm such that it is possible to handle dynamic scenarios including other ships. Another interesting topic for future work is to investigate how to incorporate regulations such as COLREGs Kufoalor et al. [2019] in the motion planning framework.

References

Thor I Fossen. *Handbook of marine craft hydrodynamics and motion control*. John Wiley & Sons, 2011.

Tristan Perez. *Ship motion control: course keeping and roll stabilisation using rudder and fins*. Springer Science & Business Media, 2006.

Aleksander Veksler et al. Dynamic positioning with model predictive control. *IEEE Trans. Control Syst. Technol.*, 24 (4):1340–1353, 2016.

Lúcia Moreira, Thor I Fossen, and C Guedes Soares. Path following control system for a tanker ship model. *Ocean Engineering*, 34(14-15):2074–2085, 2007.

Andreas B Martinsen, Anastasios M Lekkas, and Sebastien Gros. Autonomous docking using direct optimal control. In *Proc. 12th IFAC Conf. Control Appl. in Marine Systems, Robotics, and Veh. (CAMS)*, volume 52, pages 97–102. IFAC, 2019.

Zhe Du et al. Motion planning for unmanned surface vehicle based on trajectory unit. *Ocean Engineering*, 151:46–56, 2018.

Jungwook Han et al. Autonomous collision detection and avoidance for ARAGON USV: Development and field tests. *J. Field. Robot.*, 2020.

DKM Kufoalor et al. Autonomous maritime collision avoidance: Field verification of autonomous surface vehicle behavior in challenging scenarios. *J. Field. Robot.*, 2019.

Kristoffer Bergman, Oskar Ljungqvist, and Daniel Axehill. Improved optimization of motion primitives for motion planning in state lattices. In *2019 IEEE Intell. Veh. Symp. (IV)*, June 2019.

Kristoffer Bergman, Oskar Ljungqvist, and Daniel Axehill. Improved path planning by tightly combining lattice-based path planning and numerical optimal control. *Accepted for publ. in IEEE Trans. Intell. Veh.*. Pre-print available at arXiv: `https://arxiv.org/abs/1903.07900`, a.

Kristoffer Bergman et al. An optimization-based receding horizon planning algorithm. *Accepted for publ. at 21st IFAC World Congress*, 2020, b.

Mihail Pivtoraiko, Ross A Knepper, and Alonzo Kelly. Differentially constrained mobile robot motion planning in state lattices. *J. Field. Robot.*, 26(3):308–333, 2009.

Olov Andersson et al. Receding-horizon lattice-based motion planning with dynamic obstacle avoidance. In *Proc. 57th IEEE Conf. Decis. Control*, 2018.

O. Ljungqvist et al. A path planning and path-following control framework for a general 2-trailer with a car-like tractor. *J. Field. Robot.*, 36(8):1345–1377, 2019.

M. Likhachev and D. Ferguson. Planning long dynamically feasible maneuvers for autonomous vehicles. *The Int. J. Robot. Res.*, 28(8):933–945, 2009.

Marcello Cirillo, Tansel Uras, and Sven Koenig. A lattice-based approach to multi-robot motion planning for non-holonomic vehicles. In *Internat. Conf. on Intell. Robots and Systems (IROS)*, pages 232–239. IEEE, 2014.

Henrik Andreasson et al. Fast, continuous state path smoothing to improve navigation accuracy. In *IEEE Internat. Conf. on Robot. and Autom. (ICRA)*, pages 662–669. IEEE, 2015.

Sikang Liu et al. Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments. *IEEE Robot. and Autom. Lett.*, 2(3):1688–1695, 2017.

Tobias Schoels et al. CIAO*: MPC-based safe motion planning in predictable dynamic environments. *Accepted for publ. at 21st IFAC World Congress*, 2020.

Steven M LaValle. *Planning Algorithms*. Cambridge University Press, Cambridge, UK, 2006.

Edward M Lewandowski. *The dynamics of marine craft: maneuvering and seakeeping*, volume 22. World scientific, 2004.

Mihail Pivtoraiko and Alonzo Kelly. Efficient constrained path planning via search in state lattices. In *Internat. Sym. on Artificial Intell., Robot., and Autom. in Space*, pages 1–7, 2005.

Dave Ferguson and Maxim Likachev. Efficiently using cost maps for planning complex maneuvers. In *Proc. of Internat. Conf. on Robot. and Autom. Workshop on Planning with Cost Maps*. IEEE, 2008.

Ross A Knepper and Alonzo Kelly. High performance state lattice planning using heuristic look-up tables. In *2006 IEEE/RSJ Inter. Conf. on Intell. Robots and Systems*, pages 3375–3380, 2006.

Moritz Diehl et al. Fast direct multiple shooting algorithms for optimal robot control. In *Fast motions in biomechanics and robotics*, pages 65–93. Springer, 2006.

Alexander Bärlund et al. Nonlinear MPC for combined motion control and thrust allocation of ships. *Accepted for publ. at 21st IFAC World Congress*, 2020.

Joel A E Andersson et al. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, In Press, 2018.

Andreas Wächter and Lorenz T Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming*, 106(1):25–57, 2006.

Tristan Perez et al. An overview of the marine systems simulator (MSS): A simulink toolbox for marine control systems. *Modeling, Identification and Control*, 27(4):259–275, 2006.